Studio Nx

Backend(FastAPI/Python) 57 — get_membership() · require_role()

2026-10-08

Backend(FastAPI/Python) 57 — `get_membership()` · `require_role()` (프로젝트 권한 검사)

항목	내용
파일	`studionx/backend/app/services/project_service.py`
작업	P1-13 프로젝트 API — 모든 프로젝트 API 의 첫 검사 · PRD 01 §5 권한 규칙
작성	Claude (AI) · 2026-10-08 · 사용자 검토
이전 · 다음	56 프로젝트 요청 스키마 · 58 `to_item` · `list_projects`

1. 하는 일

- `get_membership`: 프로젝트 번호로 프로젝트와 **내 멤버 행**을 함께 찾는다. 접근할 수 없으면 404.
- `require_role`: 그 멤버 행의 역할이 허용 목록에 있는지 본다. 없으면 403.

상황	결과
내가 멤버인 프로젝트	`(project, member)` 돌려줌
없는 번호	404 `"Project not found."`
남의 프로젝트 (내가 멤버가 아님)	404 `"Project not found."` — 403 아님 (사용자 결정 A)
삭제된 프로젝트 (`deleted_at` 있음)	404 — 주인이어도
VIEWER 가 수정 · EDITOR 가 삭제	403 `"You don't have permission."` (`require_role`)

2. 언제 쓰나

호출하는 곳	쓰는 것
`GET /projects/{id}` 등 프로젝트 하나를 다루는 라우터 (62)	`get_membership` — 멤버면 누구나 조회
`update_project` (60)	`require_role(member, "OWNER", "EDITOR")`
`delete_project` (61)	`require_role(member, "OWNER")`
시퀀서 · 협업 API (Phase 2 · 6)	같은 두 함수를 그대로 씀

역할(`project_members.role`)은 OWNER · EDITOR · VIEWER 세 가지. Phase 1 에서는 만든 사람의 OWNER 한 행만 생긴다 (초대는 Phase 6).

3. 코드

import (파일 맨 위)

```
from sqlalchemy.orm import Session

from app.core.errors import AppError, ErrorCode
from app.models import Project, ProjectMember, User
from app.schemas.project import (
```

```
    ProjectCreateRequest,
    ProjectItem,
    ProjectListResponse,
    ProjectSort,
    ProjectUpdateRequest,
)
```

`AppError` · `ErrorCode` 줄을 이번에 추가했다. 두 함수는 `Session` · `AppError` · `ErrorCode` · `Project` · `ProjectMember` · `User` 를 쓴다.

함수

```
def get_membership(db: Session, project_id: int, user: User) -> tuple[Project, ProjectMember]:
    project = db.get(Project, project_id)
    member = None
    if project is not None and project.deleted_at is None:
        member = db.get(ProjectMember, (project_id, user.user_id))
    if member is None:
        raise AppError(ErrorCode.NOT_FOUND, 404, "Project not found.")
    return project, member


def require_role(member: ProjectMember, *roles: str) -> None:
    if member.role not in roles:
        raise AppError(ErrorCode.FORBIDDEN, 403, "You don't have permission.")
```

문법 풀이

import 문법	뜻
`from app.core.errors import AppError, ErrorCode`	우리 오류 클래스 · 오류 코드 목록 (P1-03). `AppError` 를 던지면 처리기가 `{"error": {...}}` JSON 으로 바꿈

식	뜻
`db.get(Project, project_id)`	기본 키로 한 행 찾기. 없으면 `None`
`db.get(ProjectMember, (project_id, user.user_id))`	기본 키가 두 칸(복합 키)이면 **튜플**로 넘김
`-> tuple[Project, ProjectMember]`	두 값을 함께 돌려준다는 표시. 부르는 쪽은 `project, member = get_membership(...)`
`*roles: str`	인자를 몇 개든 받아 튜플로 모음. `require_role(m, "OWNER", "EDITOR")` → `roles == ("OWNER", "EDITOR")`
`member.role not in roles`	역할은 `MemberRole`(StrEnum) 이라 문자열 `"OWNER"` 와 같다고 비교됨

4. 흐름

1. 번호로 프로젝트를 찾는다.
2. 있고 삭제되지 않았을 때만 `(프로젝트 번호, 내 회원 번호)` 로 멤버 행을 찾는다.
3. 멤버 행이 없으면 — 이유(없음 · 삭제됨 · 남의 것)와 상관없이 — 404.
4. (수정 · 삭제일 때) `require_role` 로 역할 확인 → 모자라면 403.

5. 왜 이렇게

선택	이유
남의 프로젝트도 **404**	403 이면 "그 번호의 프로젝트는 있다"는 것이 드러난다. 번호를 1, 2, 3... 넣어 보며 남의 프로젝트가 있는지 알아낼 수 없게 (사용자 결정 A, 템플릿 권장)
세 경우를 `member is None` 하나로	없음 · 삭제됨 · 남의 것이 모두 같은 응답이어야 하므로 오류를 한 곳에서만 던짐
삭제된 프로젝트는 멤버 행을 찾지 않음	쓸데없는 쿼리를 줄이고, 주인이어도 삭제된 것은 못 보게 (소프트 삭제 61)
역할 부족은 **403**	이미 멤버라 프로젝트가 있다는 것을 아는 사람 → 숨길 것이 없고, "권한 없음"이 더 정확
두 함수로 나눔	조회는 멤버면 충분, 수정 · 삭제만 역할을 봄 — 필요한 곳에서만 `require_role` 을 한 줄 더 부름

6. 시험 (`tests/test_project_service.py` 새 파일 · 12개)

시험	확인한 것
주인	`(project, member)`, 역할 OWNER
EDITOR 멤버	멤버 행을 찾아 역할 EDITOR
없는 번호	404 `NOT_FOUND` "Project not found."
남의 프로젝트	404 (403 아님)
삭제된 프로젝트	주인이어도 404
`require_role` 통과 4가지	OWNER→OWNER, OWNER·EDITOR 에 OWNER · EDITOR, 셋 다 허용에 VIEWER
`require_role` 거부 3가지	EDITOR→OWNER 만, VIEWER→OWNER 만, VIEWER→OWNER·EDITOR → 403 `FORBIDDEN`

전체 시험 450개 통과, `ruff` 검사 통과.